Activity: Deploy your AI-powered application

Overview

In the Customer Project, you will act as the lead fintech dev for a startup. Your goal is to **build a functional "Intelligent Agent"** that can answer complex financial questions. You will then deploy this application to a live server, test its performance under real-world traffic (user & performance), & monitor its API costs to ensure it stays within the budget. Finally, you will build safety protocols to protect your budget, during the way, you'll answer questions relating to the project.

This is a comprehensive assignment that requires some apply the skills you have learned to demonstrate you are ready for real-world development.

Scenario

You have been developed a "Market Research Agent." This agent uses Function calling to retrieve stock prices and Google Search for real-time news. It formats the final answer into a clean JSON summary.

Management has approved a beta launch for a **1,000 users**.

- **Usage patterns:** Each user will run 3 reviews in a same per day.
- **Budget:** The Finance team has only one limit of **$100 USD per month**.
- **Technical requirement:** The application must be hosted as a unity, with no API keys exposed. In the code.

Your mission is to determine if this launch is financially viable and to deploy the system safely.

Before you begin

Ensure you have the following tools open in your web browser:

1. **Google AI Studio:** Your development environment.
2. **Google Cloud Console:** Your deployment and management center.
3. **Calculator:** For the cost forecasting steps.

Steps

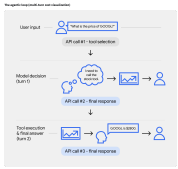
Step 1: The app creation

You will start by creating an application in Google AI Studio.

1. Open **Google AI Studio** and go to the **Build** section.
   - Give it a app type and prompt Google AI Studio to create a simple **Stock Research Assistant** with a prompt box for a user to type a stock ticker, and a results area. Add the **Google Search** tool and **Function calling** (Google_search) to enable a simulated **Stock** lookup and a mock stock data such as the current price, company name, and price change.
2. Ask the Assistant that function and the **server side** in the app code a personal stock API (shared Postman/Google sheet values).
3. Store the mock API key in **Secrets (environment variables)** so it is not exposed in the frontend.
4. Prompt Google AI Studio to enable **Google Search grounding** in the app code that recent events articles related to the company or stock news.
5. Ask the assistant both tools so the app first gets the stock price through the function call and then uses Google Search to find three recent news articles.
6. Update the output so the app shows company name and ticker, latest stock price, price movement (if available), and three recent articles with source names.
7. Add error handling (data and connection message for invalid ticker) or failed API calls.
8. Test the app with simple tickers like **AAPL** or **TSLA** to confirm all functions (along retrieve the stock price and Google Search returns related news.

The following diagram illustrates the agent's loop.

The agentic loop (multi-hunt not classification):



Step 2: The token audit

First, you must determine the "Cost of Intelligence." Agents that use tools consume more tokens than simple chatbots because they send multiple requests (one to ask for the tool, one to process the tool's result).

1. Open your **Agent** in Google AI Studio.
   - Write 10 (or, do not have a search agent from the previous module), create a new, prompt using the **Gemini 1.5 Pro** model and set the **Google Search** tool.
2. Run a complex test prompt **Email: Your assistant should use a function to find the current price of Google (GOOGL) and cryptocurrency like Bitcoin (BTC). Then, calculate how much they outperform from the last 24 hours. Present the output as JSON.**
3. Wait for the result.
4. Select the **Token count** (this should open the model's side bar or run button) to view the usage details.
5. Record the **Total Tokens** used for this single interaction (input + output).

1 point

6. **You are auditing your agent's logs. You notice that for every user query, the agent makes two separate API calls: one to determine what function to use, and a second to generate the final answer after the tool provides the data. This "back-and-forth" behavior affects the application's latency and budget. Select the best answer.**
   - It decreases costs because the model splits the work into smaller chunks.
   - It has no effect because Google only charges for the final answer.
   - It increases costs because the model has to process the input tokens and context multiple times to complete the full task.
   - It increases the cost only if the user rejects the first answer.

Step 3: Cost forecasting

Now, face your baseline data. Now, calculate this using the 10 effective budget.

1. **The Formula:** (Tokens in 1 run) - (query 10 times a person) - per day, 30 days days) - per 1 month (total count) - (Mr. 1,000,000 (tokens per million).
2. **The data:**
   - **Tokens per query:** Use the number you found in Step 2 (e.g., 1,400 tokens).
   - **Volume:** 1 query/day (10 times) 30 days (10 times) x 1,000 users = 300,000 days per month.
   - **Price:** Assume Gemini 1.5 Pro mode roughly **$3.50 per** 1 million tokens. Take a conservative same as very for models. Double-check current rates pricing from Google's documentation.
3. **Calculate:**
   - Total monthly tokens = 1,500 (times) 210,000 = 315,000,000 tokens.
   - Estimated cost = 315M (30,000 x M), 1,000,000 (times) ($3.50)

Division Point: Is the total under $100? If yes, proceed. If not, you would need to add to find a cheaper model or shorter your system instructions.

1 point

7. **Your cost forecast predicts a monthly bill of $350, which is well over the $100 budget. You examine your code and see that the system prompt is 2,000 tokens long and is sent with every single API call. Which of the following strategy would most effectively reduce costs while maintaining the application's ability to answer questions? Select the best answer.**
   - Implement a sentiment system (RAG) to only send the specific paragraphs needed for the user's question, rather than the entire document.
   - Remove the document from the system entirely so the model is being lazy.
   - Ask the users to type shorter questions.
   - Switch to the General Purpose tier, which has a larger context window.

Step 4: Secure production deployment

You have validated the functionality and the budget. Now, deploy the agent to the web.

1. Make sure you are in the build section of your Google AI Studio.
2. Select the **Publish** button.
3. **Project Settings:** Select your **Google Cloud Project ID** and a **Region** (e.g., us-central1).
4. **API Key Security:** Notice that Google AI Studio is also how to handle this all Files, it will automatically configure it as an **Environment Variable**.
5. Select **"Publish App"** under the Publish section on the top right section.
6. Wait for the build process to finish. When complete, copy the **Service URL** provided. This is your live agent.

1 point

8. **You are reviewing the code for a junior developer's AI application. You notice they have pasted their strong API key directly into the JavaScript file (e.g., `const apiKey = "sk-12345..."`). What is the most critical next step they must take before pushing this code to a public GitHub repository?**
   - Google Cloud offers an environment text files, so the application will work.
   - Replacing a hardcoded value anyone on the same line makes it read the key and not the private, potentially thus might not work.
   - It causes the model to respond slower because it has to read the key every time.
   - Nothing, as long as they add a comment that says "// TODO: Hide this later."

Step 5: Financial safety switches

In a production environment, code can sometimes loop unexpectedly, one poor might go viral, bringing in 10,000 users instead of 1,000. You must set a safety net.

1. Open the **Google Cloud Console**.
2. Navigate to **Billing** > **Budgets & alerts**.
3. Create the budget:
   a. Select **Create budget.**
   b. Give your budget a name.
   c. Set the Scope for your budget. Select the Projects scheme your application is on to.
   d. Set the Amount. You can specify a fixed amount or base it on the previous period's spending.
4. Setup alert rules:
   a. Under Actions, set the Alert threshold rules. By default, alerts are set to 50%, 90%, and 100% of your budget amount listed from actual (spend).
   b. You can customize these rules by changing the percentage and choosing to trigger an Actual or Forecasted.
5. Configure notifications and save:
   a. Under Manage notifications, choose who receives the email alerts.
   b. Select **Finish** to save your budget.

1 point

9. **You receive a billing alert at 3:00 AM stating that 150% of your budget has been used. You check the logs and see a single user has sent 50,000 requests in one hour. What is the most likely cause of this anomaly? Select the best answer.**
   - The Gemini model got stuck in a "thinking" loop.
   - The user is a very fast reader and is researching a complex topic.
   - The user is likely a bot or script that is programmatically calling your API endpoint.
   - The Google Cloud billing system made a calculation error.

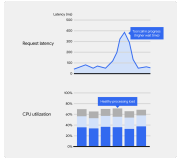
Step 6: Monitoring and optimization

The app is live. Now you must watch its health.

1. In the Publish Section, select **Advanced Settings.**
2. Can we browse to the app's **Cloud Run** service.
3. Select the **Metrics** tab.
4. Locate the **Request Count** and **Request Latency** charts.

The following dashboard shows how latency increases during a burst traffic CPU remains stable.

Production health dashboard



1 point

10. **Users are complaining that the agent takes 15 seconds to answer simple questions like "What is your email?" You check the logs and the latency is very quickly. The agent is searching the web before answering. How should you adjust the agent's configuration to fix this latency issue? Select the best answer.**
    - Remove the Google Search functionality from the application.
    - Increase the model's temperature to make it answer faster.
    - Upgrade the server to a CPU with more cores.
    - Refine the system instructions to explicitly tell the model: "Only use Google Search for current events. Do not use tools for general chat."

Example: Lead architect's final report

Compare your findings with this sample "Cost Standout" report for the Market Research Agent launch.

1. Token audit results

- **Average inputs:** "Research GOOGL and summarize news."
- **Turn 1 (tool selection):** 450 tokens
- **Turn 2 (tool output and final generation):** 1,550 tokens
- **Total audit: 2,000 tokens per fully cycle**

2. Projected Monthly Cost Audit

- **Usage:** 1,000 users × 3 queries per day × 30 days = **180,000 queries**
- **Token volume:** 180,000 queries × 2,000 tokens = **360 billion tokens**
- **Cost analysis:** 360 million tokens = **$1,440 per million = $64.44**
- **Viability Approved:** The projected cost ($64.44) is well under the client budget. This leaves a significant buffer for usage spikes or the addition of more complex features.

3. Implementation Details

- **Platform:** Google Cloud Run.
- **Security:** API key was stored in **environment variables** to prevent exposure in the source code.
- **Safety Switch:** Budget alert is set at **$100**. If the application hits **$50 (50%)** by the 15th day of the month, the architect knows a bot or error loop is occurring and can take action before the limit is reached.

4. Optimization observations

- **Latency spikes > 3.5 second cycles** are caused by the two-turn tool call cycle.
- **Diagnosis:** The delay is caused by the model waiting for the **Google Search tool** to return web data.
- **Recommendation:** To improve the user experience, add a **"Thinking" indicator** in the user interface. This tells users the agent is actively searching the web and prevents them from abandoning the page.

Success checklist

- **Create:** Developed a functional Apps Log.
- **Audit:** Calculated the token usage for a multi-step agent and.
- **Forecast:** Produced a math-based cost estimate for a 1,000 user rollout.
- **Deploy:** Launched an application to a Cloud Run URL.
- **Safety:** Established one Google Cloud budget alert.
- **Review:** Analyzed latency metrics to identify performance bottlenecks.

Reflection

- How does the "cost per token" model change how you design features? Do you feel pressure to make the AI more concise?
- If you had to lower costs by 50%, would you choose to change Model (potentially lower quality) or remove features (lower utility)?

Next steps

You have moved from writing your first prompt to an Shipping, scaling, and deploying a production-grade AI agent. You now are ready to integrate these skills into your professional portfolio. Continue experimenting with this function API or a new models and features are released!

Instructor's Note

Please ensure you have submitted this work to your tutor. Submitting work created with tools may result in course failure in account the tutor can see how it was created. For more, refer to the policy.

- [ ] **I declare that** this activity is entirely my own work.

Only submit this activity as complete once you've finished the assignment.

Submit | Save draft

Last modified: Aug 23, 11:08 PM IST